Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

- An expression tree is a tree where the internal nodes are operators and leaves are operands
- **Algorithm**:
  - **Input** : an expression in postfix. For simplicity, the operands are assumed to be single digits.
  - **Process** : construct an expression tree
  - **Output**: find the value of the expression

© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

Phase 1:
Construct the expression tree:

Data structure used: stack of tree nodes

- Scan the input left to right
- If operand, create a tree node and push it to a stack of tree nodes.
- If operator, pop two tree nodes from the stack, create a new tree node with the operator as the key.
- Attach the popped nodes as subtrees of the new node.
- Push the new node into the stack.

© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

Phase 2 :

- Evaluate the expression tree:
- Stack should have one and only one tree node.

- On this tree, do the evaluation.
- If the node is a leaf,
  - return the operand.
- Else
  - Evaluate the left subtree
  - Evaluate the right subtree
  - Apply the operator of this node on these two results

© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

Structure:

```
#define MAXSIZE 100
struct node
{
        char ch_;
        struct node *left_;
        struct node *right_;
};
typedef struct node node_t;

struct stack
{
        node_t* key_[MAXSIZE];
        int top_;
};
typedef struct stack stack_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

interface:

```
void init_stack(stack_t *ptr_stack);
void deinit_stack();
// push modified to take care of operand and operator differently
void push_operand(stack_t *ptr_stack, char ch);
void push_operator(stack_t *ptr_stack, char ch);
// pop returns a pointer to a tree node
node_t* pop(stack_t *ptr_stack);
int is_empty(stack_t *ptr_stack);
int is_full(stack_t *ptr_stack);
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

implementation:
create_node: helper function

```
static node_t* create_node(char ch)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        if(temp == NULL)
        {
                printf("memory full\n"); exit(1);
        }
        temp->ch_ = ch;
        temp->left_ = temp->right_ = NULL;
        return temp;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

implementation:

push_operand: single char operand; could use a union of operator and operand

```
void push_operand(stack_t *ptr_stack, char ch)
{
        if(! is_full(ptr_stack))
        {
                node_t* temp = create_node(ch);
                ptr_stack->key_[++ptr_stack->top_] = temp;
        }
        else
        {
                printf("stack full; cannot push\n");
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

implementation:

```
void push_operator(stack_t *ptr_stack, char ch)
{
        if(! is_full(ptr_stack))
        {
                node_t* temp = create_node(ch);
                temp->right_ = pop(ptr_stack);
                temp->left_ = pop(ptr_stack);
                ptr_stack->key_[++ptr_stack->top_] = temp;
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

implementation:

```
node_t* pop(stack_t *ptr_stack)
{
        if(! is_empty(ptr_stack))
        {
                return ptr_stack->key_[ptr_stack->top_--];
        }
        else
        {
                printf("stack empty; cannot pop\n");
                return '\0';
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

implementation: stack should have one element - root of the expression tree.

- Pass this as argument to eval_recursive.
- If leaf, then its an operand – find and return the numeric value of the character.
- Evaluate left then evaluate right
- Based on the operator, combine the two results.

```
int eval_recursive(node_t* root)
{
   int left, right;
   if (root->left_ == NULL &&
                  root->right_ == NULL)
      return root->ch_ - '0';

   left = eval_recursive(root->left_);
   right = eval_recursive(root->right_);
```

```
switch (root->ch_)
   {
      case '+': return left + right;
      case '-': return left - right;
      case '*': return left * right;
      case '/': return left / right;
   }
    return 0;
}
 int eval(stack_t *ptr_stack)
{
return eval_recursive(pop(ptr_stack));
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)